Teckashy hackathon. Agriculture and Climate track

72:00:00

72 Ghante

When a hailstorm hits, an insured farmer in India has 72 hours to report the crop loss. Today the system waits for the farmer to come forward. 72 Ghante goes to the farmer: AI agents phone every affected farmer in Hindi, collect what happened and a photo, check the claim, and file the loss report before the deadline.

The project explained, for someone who has never seen it

1 The problem
2 Who it is for
3 The solution, step by step
4 Workflow diagram
5 Why it is novel
6 The six components and how each is used
7 System architecture
8 Inside the phone call
9 How a claim is checked
10 The demo and the screens
11 Measured results and testing
12 Safety, honesty and zero cost
13 Limits and the path to a real district
14 Glossary and sources

[Team name]. Version of 8 October 2026. All people, policies and the insurer portal shown are fictional or simulated, and labelled so.

At a glance

The whole project on one page

"72 Ghante" means "72 hours" in Hindi. It is a working system, built in four days on free tools, that turns a crop-insurance deadline the farmer usually misses into a phone call the farmer only has to answer.

The problem in three sentences

Under India's crop insurance scheme (PMFBY), a farmer whose field is hit by hail, a flood, a landslide, a cloudburst or a fire, or whose cut crop is ruined by rain while drying, must report the loss within 72 hours. If the report is late, the claim can be rejected. The burden is entirely on the farmer, in the three worst days of the season, often with a basic phone, no internet and no idea the rule exists.

What 72 Ghante does

1. **Notices the storm** from the weather record, by itself.
2. **Finds every insured farmer in its path** and decides, with fast small models, who to call first.
3. **Phones each farmer in Hindi.** The agent says it is an automated assistant and asks seven simple questions: is this Ramlal ji, was there damage, is it field 214, what caused it, when, how much of the crop, and was it standing or cut.
4. **Asks for a photo,** which the farmer's phone checks by itself, even with no internet.
5. **Checks the claim** against the policy, the deadline, the weather record and the photo's location.
6. **Fills the insurer's online form like a person would,** page by page, in a real browser, and reads the reference number back to the farmer before hanging up.
7. **Shows the whole district on one screen:** 340 farmers, a 72-hour countdown, and every decision as it is made.

312/340
farmers in the demo storm get a filed report, with 68 of the 72 hours to spare

24 ms
for a small model to understand a farmer's Hindi reply (median)

32 s
for the browser agent to fill the three-page insurer form; correct 5 times out of 5

What makes it different

Every existing channel (helpline, app, bank, office, chatbot) waits for the farmer and stops at information. 72 Ghante is the first, as far as we know, to **start the conversation itself when the storm happens** and to **finish the task**: the output is a filed report with a reference number, not advice.

What is real and what is simulated

One farmer in the demo is a real person on a real phone, in a real Hindi conversation. The other 339 farmers are scripted, and the screen says so ("Simulated farmers: 339. Live: 1."), but their cases pass through the same models, rules and tool servers. The insurer portal is our own clearly labelled sandbox with fictional people; nothing is filed anywhere real. Nothing was paid for: every tool is open source or a free tier.

1

The problem: a 72-hour deadline that waits for the farmer

The rule

India's crop insurance scheme is called **PMFBY** (Pradhan Mantri Fasal Bima Yojana). Most losses are assessed for a whole area at the end of the season. But two kinds of loss are assessed farm by farm, and only if the farmer reports them:

- **Localized calamities** on one farm: hailstorm, landslide, inundation (flooding), cloudburst and natural fire.
- **Post-harvest losses**: a crop cut and left to dry in the field, then ruined by hail, cyclone or unseasonal rain.

For these, the farmer must inform the insurer **within 72 hours** of the loss. The government told Parliament in December 2021 that the window had been 48 hours until 2018 and was extended to 72 hours "keeping the convenience of farmers in mind". News coverage in March 2026 still describes the same 72-hour rule and warns that late reports can be rejected.

Why the deadline is hard to meet

- **The farmer has to know the rule.** Many do not, or learn about it after the window has closed.
- **The farmer has to act first,** in the three days right after a storm, while dealing with a ruined field.
- **The channels assume resources:** a smartphone and data for the app, literacy for the forms, a trip to a bank or office, or getting through a helpline queue on the day everyone in the district is calling.
- **Power and network often fail** exactly when storms hit.

> **A documented case.** In October 2020 a soybean farmer in Osmanabad, Maharashtra lost 60 to 70 quintals of crop to two weeks of rain. His claim was rejected because he had not informed the insurer within 72 hours; he said there was no power to charge his phone. (People's Archive of Rural India, 8 September 2021.)

What exists today, and the gap

Channel	What it does	What it leaves to the farmer
Helpline 14447 (Krishi Rakshak)	A farmer can call and report a loss	Knowing the number and the deadline, calling first, waiting in a queue after a storm
Crop Insurance app	Self-service loss reporting	A smartphone, data, reading and familiarity with the app
Bank branch or agriculture office	Accepts the intimation	A trip, in the days after a storm
Kisan e-Mitra voice bot	Answers questions about a government scheme by voice	It informs; it files nothing

The gap: nothing calls the farmer first, and nothing files the report for them. Every existing tool waits for the farmer and stops at information.

2

Who it is for

The farmer (the user)

Our demo farmer is Ramlal Verma, 52, a wheat farmer in Sehore, Madhya Pradesh, with 1.8 hectares insured. He has a basic phone, speaks Hindi, reads slowly and has never opened the insurance app. He needs someone to call him, ask simple questions in Hindi, and do the paperwork. (Ramlal is a fictional persona.)

The district agriculture officer

After one storm, hundreds of reports arrive by phone and on paper, many late or incomplete. The officer needs one screen showing who was hit, who has been reached, who has been filed and who still needs a person.

The insurer's claims desk

Reports arrive with missing fields and no evidence, and disputes follow. The desk needs complete, consistent reports with a photo, location, time and the weather record attached.

The buyer

In a real deployment the caller would be the insurer or the state agriculture department: the organisations the farmer already enrolled with. The demo speaks to both: the phone call is for the farmer, the map is for the officer.

The pitch in one line

"After a hailstorm, a farmer has 72 hours to tell the insurer. Today the system waits for the farmer to find it. We reverse that: the system finds the farmer."

3

The solution, step by step

This is what happens in the demo, which replays a hailstorm over 12 villages of Sehore district at 15:30 on 12 March. A scenario clock runs fast (about 180 times real time) so that the storm and three days of follow-up fit into a few minutes on stage.

1. **The storm is noticed.** The *Sentinel* agent asks the weather tool server for storm events and finds the hailstorm: its centre, radius, start time and how hard each village was hit (severe, moderate or light).
2. **Affected farmers are found.** The *Dispatcher* agent asks the farmer-registry tool server for every insured farmer in those villages: 340 farmers. Each gets a case with a deadline of exactly 72 hours after the storm.
3. **They are ranked.** A small fast model (System One, decision *TRIAGE*) reads a one-line report for each farm ("Hail intensity at this village was severe. The farm is 1 km from the storm core. Crop: wheat, close to harvest.") and gives the chance that the farm was damaged. The worst-hit, nearest farms are called first.
4. **Calls go out,** eight at a time, only between 08:00 and 20:00. A farmer who does not answer is tried again 30 minutes later, up to three times, then marked "not reachable" for a person to follow up.
5. **The live call.** The presenter presses H and the real phone rings. The *Intake* voice agent greets Ramlal by name in Hindi, says it is an automated assistant calling about today's hail, and asks its seven questions. Simple yes or no answers are understood by a small model in about 25 milliseconds; longer answers ("the hail flattened the whole wheat") go to a language model that only extracts the facts.
6. **The photo.** The agent asks for a photo of the damaged crop. The farmer presses one button; the phone's own small vision model labels it ("hail damage") in a fraction of a second, with no server and no internet, and sends it with its location and time.
7. **Read-back and consent.** The agent reads every detail back in Hindi and asks for permission to file. Nothing is filed without "yes".
8. **The claim is checked.** The pipeline looks up the weather at the photo's exact spot and time, measures how far the photo was taken from the insured plot, runs nine eligibility rules, and asks a small model for a second opinion on whether the statement, photo and weather tell one story.
9. **The form is filed.** A browser agent opens the insurer's three-page web form (our sandbox) and fills it like a person: it reads each page, types, selects, uploads the photo, ticks the declaration and submits. If it fails or is too slow, a scripted fill finishes the job, and the screen says so.
10. **The farmer is told.** While Ramlal is still on the line, the agent reads out the reference number ("FCS-000066"), and a message card with the number arrives on his phone.
11. **The district finishes.** The presenter presses P and the other 339 simulated cases continue. The run ends with: "312 of 340 claims filed. 68 hours left. 16 need follow-up." The 28 not filed are shown honestly: 11 not reachable, 9 with no damage, 5 handed to a person, 3 not eligible.

4

Workflow diagram

How one storm flows through the system, from the weather record to a filed report. Coloured boxes show which of the six components does the work.

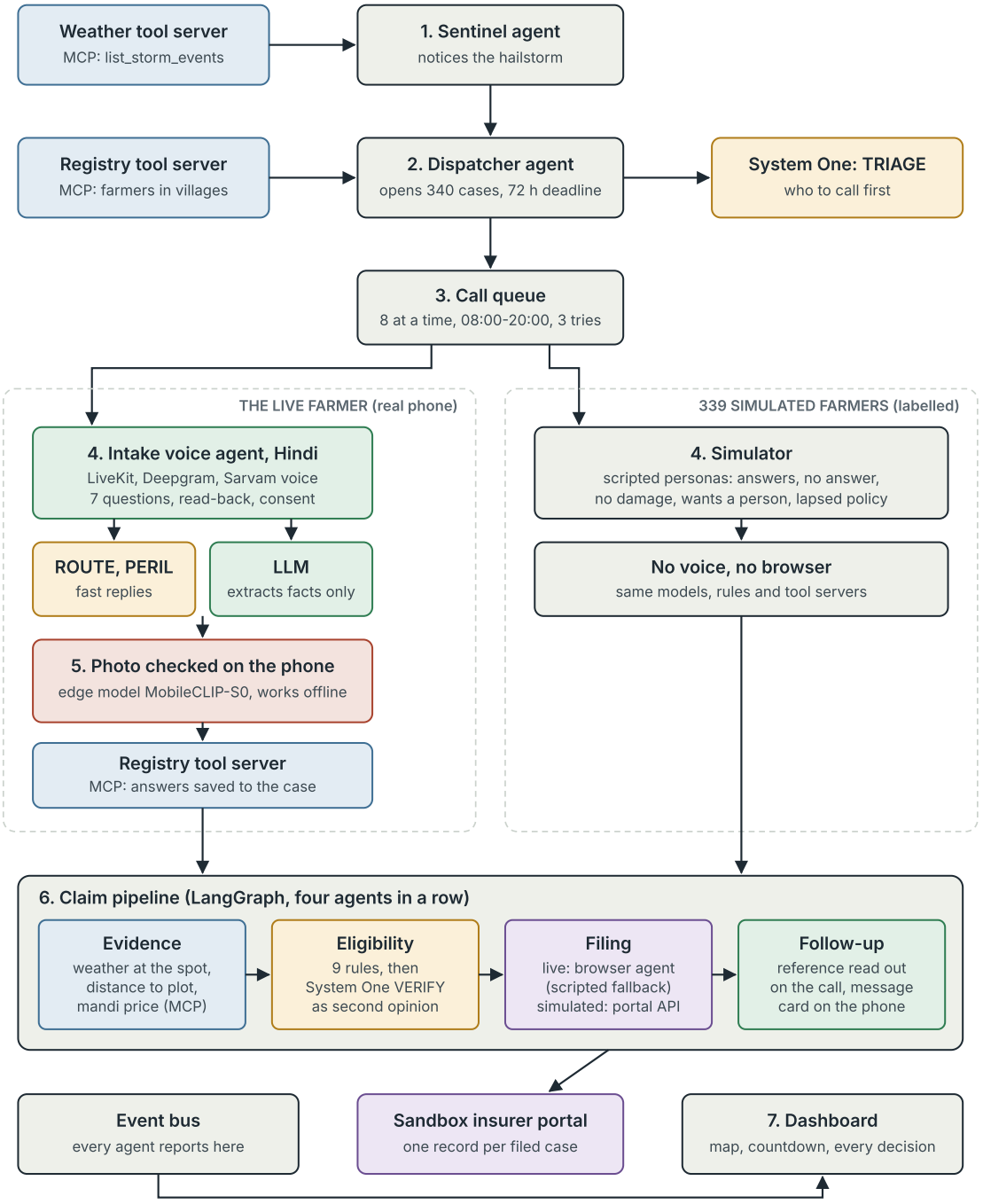


Figure 1. Workflow. Blue: MCP tool servers. Gold: System One decision models. Green: voice. Red: edge model on the phone. Purple: browser and portal. Grey: orchestration agents. Every box reports what it does to the event bus, and the dashboard draws those events.

5

Why it is novel

Each part on its own exists somewhere: voice bots, crop insurance apps, weather alerts, form-filling robots. What is new is putting them together around one deadline, in this order, for this user.

1. Proactive, not reactive

The trigger is the weather event, not the farmer. The farmer does not need to know the rule, the number or the app. The system calls within the hour of the storm being recorded.

2. Action, not advice

Existing voice bots answer questions. 72 Ghante finishes the task: the output is a filed report with a reference number the farmer hears before hanging up.

3. Evidence by default

Every report carries a photo checked on the phone, its GPS location and time, and the weather record for that exact spot. Mismatches go to a person. This makes reports harder to fake and easier to approve.

4. Built for a whole district at once

One storm means hundreds of calls on the same afternoon. Small fast models decide who to call first and understand routine replies, so the expensive large model is used only where it is needed.

5. Works around missing APIs

Most public portals have no programming interface. A browser agent uses the website the way a person would, so the system does not have to wait for an insurer to build an API.

6. Honest by design

Simulated and live are labelled everywhere; every fallback that fires is shown on the screen; the agent says it is automated, asks consent, never promises money and offers a person.

Compared with what exists

	Helpline / app / office	Information voice bots	72 Ghante
Who starts it	The farmer	The farmer	The system, when the storm is recorded
Language	Varies; forms are text	Indian languages, by voice	Hindi, by voice (other languages: configuration plus translated lines)
Needs a smartphone	App: yes	No	No for the call; a phone camera for the photo
Files the report	Only if the farmer completes it	No	Yes, with a reference number
Evidence attached	Rarely	No	Photo, location, time, weather record
Scale on one afternoon	Queues	One conversation at a time	A district, ranked and called in parallel

6

The six components and how each is used

The hackathon asks for six technologies. In 72 Ghante each one has a job the demo cannot do without; none is there for show.

---

1. Agentic browser use

A browser agent fills the insurer's form like a person

Most government and insurer portals have no API. Our **Filer** opens a real Chrome window on our sandbox insurer portal ("FasalClaim Sandbox") and an AI agent fills it: it reads each page, types the farmer's details, selects the crop and cause, uploads the photo, ticks the declaration and submits. The audience watches the form fill itself.

Built with	browser-use 0.13 driving Chrome; Groq `openai/gpt-oss-120b` as the model (free tier); one small agent per form page, so each page needs one model answer
Fallback	If the agent fails or takes longer than its time limit, a scripted fill (Playwright, by field id) finishes the same form in under a second, and the dashboard says "fallback used"
Safety	The Filer refuses any address other than the local sandbox; the agent is restricted to that one domain
Measured	5 correct out of 5, 30 to 32 seconds, 3 model calls per form; scripted fill 0.7 seconds

---

2. Ultra-low-latency voice

A natural Hindi phone conversation

The **Intake** agent talks to the farmer. To keep it fast and predictable, every question is a fixed, pre-written Hindi line, recorded once in a natural voice and played from a cache. The large language model is never asked to "talk"; it is only asked to pull facts out of what the farmer said (cause, time, share of crop lost, crop stage) and return them as data. Simple "yes" and "no" replies skip the language model entirely.

Built with	LiveKit Agents (real-time audio rooms), Deepgram nova-3 speech-to-text, Sarvam bulbul voice from a pre-built cache, local Piper voice as the free fallback, Groq for fact extraction (free tier), with a Gemini model as the backup
Speed tricks	Cached audio for every fixed line; the model connection is opened during the greeting; a second request is sent if the first is slow; the read-back cannot be interrupted, so consent is never taken from a half-heard remark
Measured	Thinking time (no audio): fast path about 25 ms; language-model path median 0.66 to 0.96 s across runs

3. System One decision models

Small, fast models make the thousands of routine decisions

"System One" means fast, intuitive decisions, as opposed to slow reasoning. One storm needs hundreds of small, bounded decisions; a large language model would be slow and costly for each. Our decision service answers four kinds, each with a probability, and the dashboard shows every one live in a "fast decisions" ticker.

Decision	Question	Model	Speed
TRIAGE	How likely is this farm damaged? (who to call first)	Laya, an open model, on a short English report	about 160-180 ms; identical reports answered once
ROUTE	Is this Hindi reply a clear yes, no, a request for a person, or something the language model must read?	Our own classifier: a multilingual sentence encoder plus a trained head	about 20 ms; 29 of 30 right in testing
PERIL	What caused the damage? (hail, flood, ...)	Same classifier	about 14 ms
VERIFY	Do the statement, photo and weather tell one story?	Laya, as a second opinion after the rules	about 220 ms

The example product named in the brief, Jev, is paid, so we used free open equivalents. If a model is slow or fails, the next one in its chain answers and the ticker marks it.

4. Edge models

The phone checks the photo by itself

During the call the farmer taps "photo" on a simple Hindi web page on their phone. A small vision model running inside the phone's browser labels the picture as hail damage, waterlogged, lodged (flattened), healthy or not a crop. If the photo shows no crop or is unclear, the phone asks for a retake on the spot. If there is no network, the photo is kept on the phone and sent automatically when the network returns.

Built with	MobileCLIP-S0 run in the browser with transformers.js; label descriptions prepared once; a service worker and a local queue for offline use
Why it matters	No server is needed, it works where the network is weak, the farmer's photo is checked instantly, and the label travels with the evidence
Measured	240 ms per photo in the laptop's browser; a phone will be slower (to be measured on the demo phone)

5. Multi-agent orchestration

Specialised agents hand work to each other

No single program does everything. Each agent has one job and reports every action to a shared event bus, which the dashboard draws.

Agent	Job
Sentinel	Watches the weather record and announces the storm
Dispatcher	Opens cases, ranks them, places calls in order, limits parallel calls, retries, respects calling hours
Intake	The voice agent on the live call
Simulator	Plays the 339 scripted farmers, clearly labelled
Evidence, Eligibility, Filing, Follow-up	The claim pipeline, built as a graph with LangGraph: each passes the case to the next, or stops with a reason

A shared scenario clock lets the presenter pause the whole district (P) while the live call happens, so the audience can follow one farmer, then resume.

6. MCP (Model Context Protocol)

Agents share tools through a standard protocol

MCP is an open standard for giving AI agents tools. Instead of each agent having its own private code for the weather or the farmer records, three tool servers offer those tools to any agent that speaks MCP.

Server	Tools	Used by
Weather	`list_storm_events`, `get_weather_at`	Sentinel; Evidence agent
Registry	`list_farmers_in_villages`, `get_farmer`, `get_policy`, `get_case`, `update_case_slots`, `set_case_status`	Dispatcher; Intake voice agent
Mandi	`get_price` (market price, for an illustrative loss estimate)	Evidence agent

Built on the official MCP Python SDK. Every tool call appears in the dashboard's "Tool calls" list with its time. In a real deployment, these servers would front the insurer's records and a real weather service without changing the agents.

7

System architecture

Everything runs on one Windows laptop and an Android phone. Each box is a separate program that can be started, stopped and tested on its own.

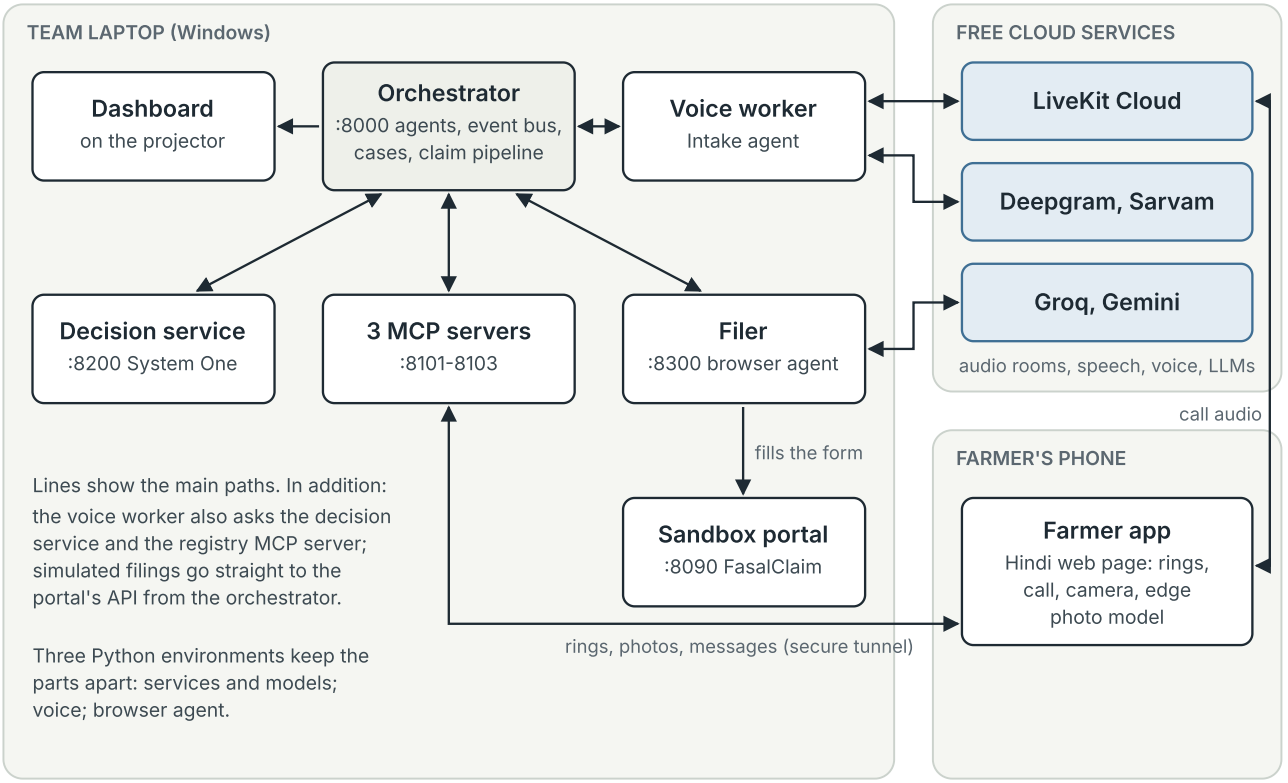


Figure 2. Programs and connections. Numbers are the ports on the laptop. The phone reaches the orchestrator through a free secure tunnel (cloudflared or localhost.run) for ringing, photos and the message card; the call's audio goes through LiveKit Cloud. The Filer's browser agent also uses Groq.

Technology list (all free)

Part	Technology
Language	Python 3.12 (services, agents, models); plain HTML, CSS and JavaScript (dashboard and farmer app)
Web services	Starlette and uvicorn; Server-Sent Events for live updates; SQLite for cases
Voice	LiveKit Agents 1.8.5 and LiveKit Cloud free tier; Deepgram (sign-up credit); Sarvam AI (sign-up credit); Piper (local, free)
Language models	Groq free tier (gpt-oss models); Google AI Studio free tier (Gemini Flash-Lite) as backup
Decision models	Laya (open model); multilingual-e5-small encoder with our trained classifier
Edge model	MobileCLIP-S0 via transformers.js in the browser
Orchestration	asyncio agents; LangGraph for the claim pipeline
Tools	Official MCP Python SDK
Browser	browser-use, Playwright, Google Chrome

8

Inside the phone call

The call follows a fixed script so that it is fast, testable and safe. The farmer can answer naturally; the agent understands, fills in what it heard, and moves on. Below, the farmer's lines are the ones used in testing.

Step	The agent asks (in Hindi)	The farmer says (example)	How it is understood
Greeting	Is this Ramlal ji? (says it is an automated assistant)	हाँ, बोल रहा हूँ। ("Yes, speaking.")	Fast path: ROUTE says "yes"
Damage	Did your wheat crop suffer damage in today's hail?	हाँ, बहुत नुकसान हुआ है।	Fast path
Plot	Is it field number 214?	हाँ, वही खेत है।	Fast path
Cause	What caused the damage?	ओले गिरे थे, गेहूँ पूरा बिछ गया।	Language model extracts the cause; PERIL labels it "hailstorm"
Time	At what time?	दोपहर करीब साढ़े तीन बजे।	Language model: 15:30
Extent	How much of the crop was lost?	आधे से ज़्यादा, करीब सत्तर प्रतिशत।	Language model: 70 %
Stage	Was the crop standing or cut?	फसल खड़ी थी।	Language model: standing

Photo	Please take a photo of the damaged crop	(presses "photo")	Edge model on the phone: hail damage
Consent	Reads all details back; may I file?	हाँ, कर दीजिए।	Fast path; filing starts
Reference	Your report is registered; the number is F, C, S ...	(listens; a message card arrives)	Read out from the portal's confirmation

One turn: fast path or language model

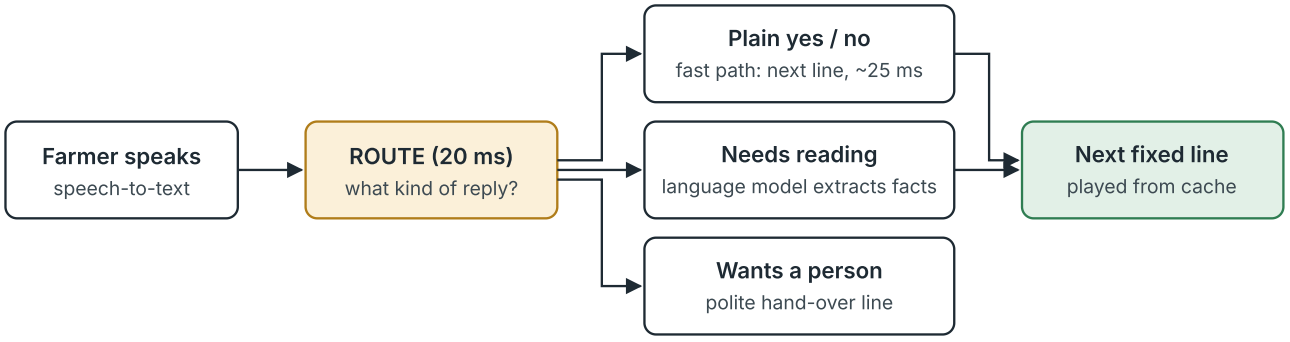


Figure 3. Every farmer reply is first classified by a small model. Only replies that need reading go to the language model, which returns facts as data; the agent's next sentence is always a pre-written line.

When the call does not go to plan

- **Wrong person** ("no" at the greeting): the agent apologises and ends; nothing is filed.
- **No damage:** the case is closed as "no damage".
- **The farmer asks for a person,** or the agent cannot understand after three tries: polite hand-over, the case shows "needs a person".
- **No consent:** nothing is filed.
- **The call drops:** the case keeps what was collected; calling again continues where it stopped.
- **Filing is slow:** the agent says it is taking longer and ends kindly; the message card still arrives when filing completes.

9

How a claim is checked

Before anything is filed, the Eligibility agent runs plain, readable rules. The small VERIFY model is asked only when every rule passes, as a second opinion. Any doubt sends the case to a person, never to an automatic rejection.

#	Check	If it fails
1	The policy is active and covers the storm date	Not eligible
2	The policy names an insured crop, and the damage is on the insured plot	Not eligible

3	The cause is covered for the crop stage (standing: hail, flood, landslide, cloudburst, fire; cut: hail, unseasonal rain, cyclone)	Not eligible
4	Reported within 72 hours	Not eligible
5	The farmer consented	A person reviews
6	There is a usable photo, not "not a crop" and not clearly "healthy"	A person reviews
7	The photo was taken within 2 km of the insured plot	A person reviews
7a	The weather record at that spot supports the cause (hail recorded for a hail claim)	A person reviews
7b	The stated time is within 3 hours of the recorded storm	A person reviews
8	VERIFY model: statement, photo and weather tell one story	A person reviews

> **Why rules first.** In testing, the small model alone judged 11 of 15 evidence cases correctly; three of its four misses were cases a number decides (a photo 15 or 40 km away, no hail on record), which rules 7 and 7a catch exactly. So numbers are judged by rules, and the model is kept for the question rules cannot answer: does the whole story hang together?

A person can approve a case the checks sent to review (key A on the dashboard); the record then says a human reviewer decided. 72 Ghante files the *intimation* (the report that a loss happened). The insurer's surveyor still assesses the damage and decides any payment.

10

The demo and the screens

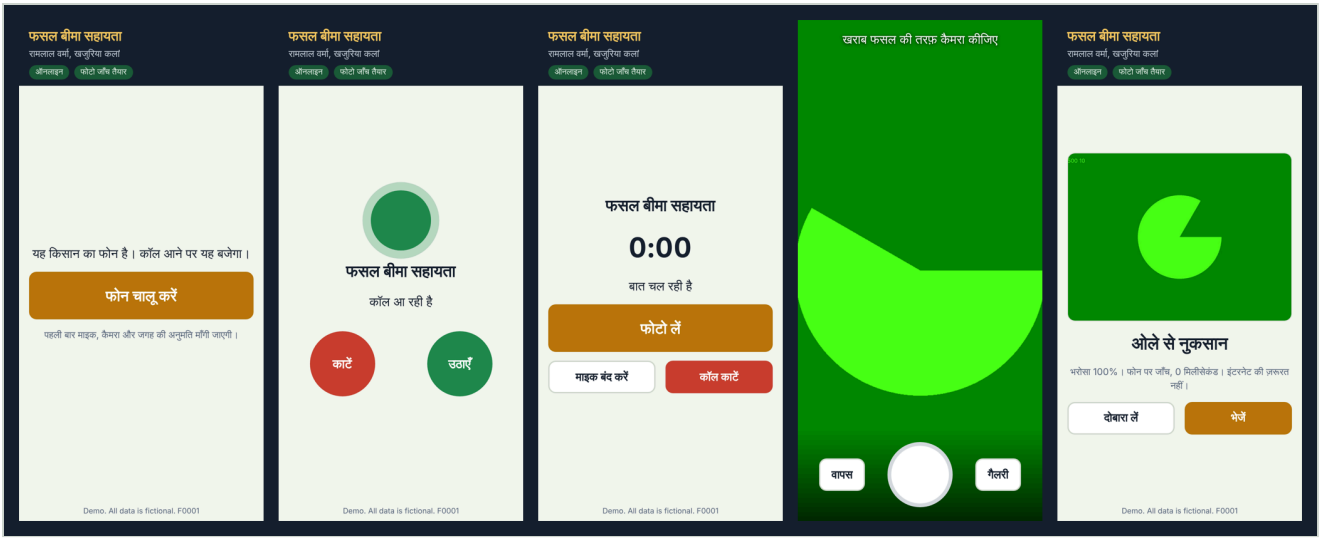


Figure 4. The farmer's phone, in Hindi: start, incoming call (answer or decline), in the call with the photo button, the camera, and the photo checked on the phone ("hail damage") with retake and send. Captured in a test browser with a stand-in camera and model; on a real phone the camera shows the field.



Figure 5. The dashboard during the live call: the district paused on the map, the live call's transcript with "fast path" or "language model" on each turn, the details collected, the countdown, the farmer counters, the decision ticker and the tool calls. Captured in an automated test with a stand-in decision model ("mock"); in the real run the ticker names the real models with their timings.

Presenter keys

Key	Action
S	Start the storm
H	Ring the live farmer (the district pauses)
P	Pause or resume the district
L	Show the whole map or the live call
A / F	Approve as reviewer / try filing again (only when needed)
R twice	Reset for the next run

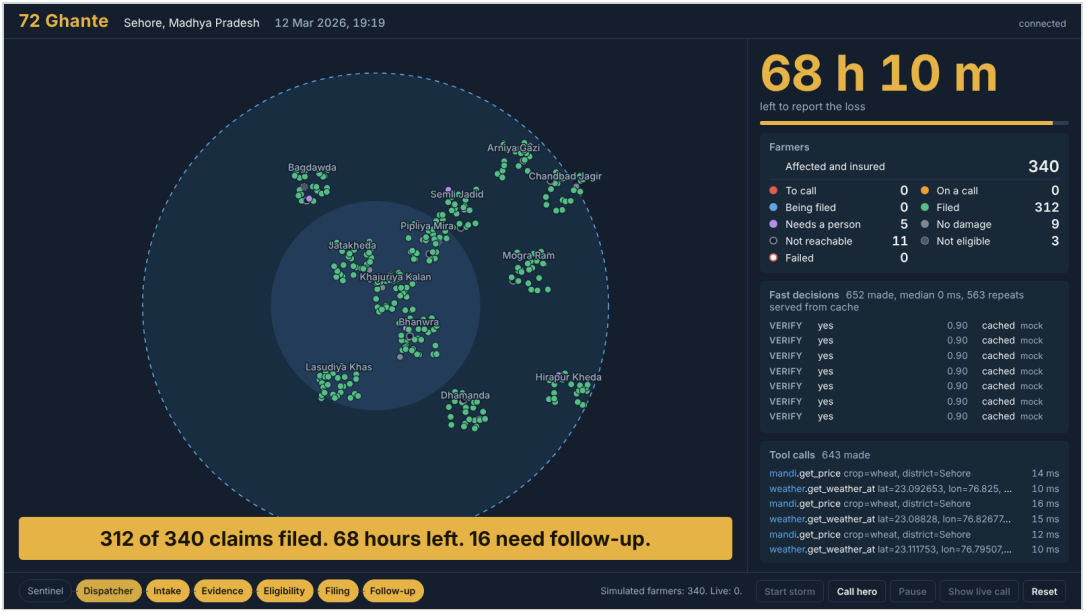


Figure 6. The end of a district run: "312 of 340 claims filed. 68 hours left. 16 need follow-up." Green dots are filed farms; the counters show every outcome. Captured in an automated test (decision model "mock", every farmer simulated).

The demo in five minutes

Time	What the audience sees
0:00	The district map, 340 grey dots, a clock at 72 hours. The problem in one sentence
0:15	S: the storm sweeps in; affected dots turn red; the ranking ticker fills
0:40	H: the district freezes; the phone rings on stage
0:50-2:15	The Hindi conversation, the photo checked on the phone, the read-back and consent
2:15-3:15	Evidence and eligibility; the browser fills the three-page form by itself
3:15	The agent reads the reference number; the message arrives on the phone
3:25-4:40	P: the district completes; the closing banner; "Simulated farmers: 339. Live: 1."

11

Measured results and testing

Every number below was measured on the team laptop on 7 and 8 October 2026. Where a number is a "thinking time", it excludes speaking and listening.

What	Measured
Understanding a short Hindi reply (ROUTE)	median 19-24 ms; 29 of 30 correct on the test set
Agent thinking time, fast path	median about 25 ms
Agent thinking time, language-model path	median 0.66-0.96 s across runs; slowest about 1.5 s
Photo check by the phone's model	240 ms per photo (in the laptop's browser)
Browser agent filing the three-page form	5 of 5 correct; 30-32 s; 3 model calls each
Scripted fill (fallback)	0.7 s
Ranking a farm (TRIAGE)	about 160-180 ms; farms whose reports read identically share one answer
District run, 340 farmers	312 filed, 11 not reachable, 9 no damage, 5 to a person, 3 not eligible; 72-77 s of real time; 68 hours left
Full rehearsal (storm, live call as text, browser filing, district)	3 of 3 in a row from a cold start; ring to reference 39-82 s (text, no speech)

How it was tested

- **Over 100 automated tests** across five development phases, from single rules to complete district runs.
- **The voice agent as text:** the real agent, models and language model, driven by the test script's Hindi lines.
- **Demo rehearsal script:** plays the whole demo from reset several times in a row and checks every beat: the district pauses during the call, nothing moves while paused, the hero is filed, the reference is read out, the totals and the closing line are exact.
- **Fallback drills:** a broken decision model, a browser agent given only 5 seconds, a wrong language-model key, the Filer and the main service killed mid-run. Each one recovers, and the screen says what happened.
- **Screen checks** in the laptop's Chrome at projector sizes 1920 × 1080 and 1366 × 768: nothing clipped, nothing scrolling.
- **A secrets check:** no API key appears anywhere outside the private settings file.

> **Still to be done by the team before judging:** five spoken runs in a row with the real phone, one run on a mobile hotspot only, the four drills that need a person, a backup video, timed rehearsals, and a check of each provider's billing page.

12

Safety, honesty and zero cost

Rules the system keeps

- The agent says it is an automated assistant, reads everything back, asks for consent before filing, never promises money or an amount, and offers a person at any point.
- Calls happen only between 08:00 and 20:00.
- Only fictional people: generated names, phone numbers starting +91 00000, application numbers starting APP-SEH.
- Only a sandbox portal, labelled as a sandbox on every page. The browser agent is restricted to it; nothing is ever filed on a real government or insurer site.
- Simulated and live are labelled everywhere, and every fallback that fires is shown on the dashboard. A recording, if used, is announced as a recording.
- Keys are kept only in a private settings file that is never shared or uploaded.

Zero cost

The team had no budget, so nothing paid was used: open-source software, free tiers and sign-up credits that need no card. A usage report counts what each free allowance has used so far. A pre-recorded audio cache means the spoken lines cost almost nothing per call (all fixed lines together used 2,059 characters, about 6 % of one free voice credit).

13

Limits and the path to a real district

What the demo does not prove yet

- **Only one farmer is real.** The others are scripted, so real-world answer rates, accents and noise at scale are not yet measured.
- **The portal is our sandbox.** Filing into a real system needs an agreement with an insurer.
- **Hindi only.** Other languages need translated lines checked by a native speaker, and a speech model that covers them.
- **How many farmers miss the 72 hours today** is not published that we could find; we have the rule and documented cases, not a national figure.
- **Free tiers have daily limits,** enough for a demo and a pilot, not a whole state.

From the demo to a district

	What changes
Who calls	The insurer or the state agriculture department, from a number farmers know
How it files	Through an insurer or government API where one exists; the browser agent where none does; a person for every flagged case
What starts it	Live weather and satellite feeds instead of a recorded storm
Phones	Real telephone calls to basic phones (a paid telephony service); the photo step for those with a camera phone
First step	One district, one insurer, one season: count the losses reported within 72 hours, against the season before

14

Glossary and sources

PMFBY
Pradhan Mantri Fasal Bima Yojana, India's national crop insurance scheme.

Intimation
The farmer's report to the insurer that a loss happened. It starts the claim; it is not the payment.

Khasra number
The land record number that identifies a plot (Ramlal's is 214).

Mandi
A wholesale agricultural market; its prices give an illustrative loss value.

Agent
A program that takes actions toward a goal, here often with an AI model deciding the next step.

LLM (large language model)
A large AI model that reads and writes text. Here it only extracts facts from the farmer's words, and drives the browser agent.

System One model
A small, fast model that makes one narrow decision with a probability, like a quick instinct, as opposed to slow reasoning.

Edge model
A model that runs on the user's own device (here the phone's browser), with no server.

MCP (Model Context Protocol)
An open standard for giving AI agents tools, so any agent can use the same weather, registry and market tools.

Orchestration
Coordinating several agents so each does one job and passes work on.

Fast path
A reply understood by the small model alone, without the language model, for speed.

Fallback
A simpler method that takes over when the main one fails, always shown on screen.

Sandbox
A copy made for testing. Our insurer portal is a sandbox with fictional data.

Sources

- Lok Sabha, Unstarred Question 3866, answered 21 December 2021 (72-hour intimation; extended from 48 hours in 2018). eparlib.nic.in/bitstream/123456789/977813/1/AU3866.pdf
- People's Archive of Rural India, "In Osmanabad, crop insurance, no assurance", Parth M.N., 8 September 2021. ruralindiaonline.org
- Goodreturns, "PMFBY rules require farmers to report hailstorm crop damage within 72 hours", 22 March 2026. goodreturns.in
- SBI General Insurance, PMFBY claim process (channels, helpline 14447). sbigeneral.in/claimprocess
- Wadhwani AI, Kisan e-Mitra. wadhwaniai.org
- Project measurements: context/Phase0Results.md to Phase5Results.md in the project folder.